TECHNICAL REVIEW

AsymptoticAnalysis

After the third review wave

Incremental code review, native Wolfram Language comparison, and Mathics semantic audit

Repository	VladimirReshetnikov/Asymptotic
Reviewed revision	`7d1bc832895c`
Review date	September 9, 2026, Pacific time
Prior-review boundary	27 reports; 167 indexed finding entries
Mathics target inspected	Mathics3 10.0.1 / SymPy 1.14.0

Principal conclusion

The new portability layer deserves the same mathematical scrutiny as the series engines. An exact special-function conversion can invalidate a retained real branch, and an acceptance runner can lose evidence of a source change it already detected.

This report separates source analysis, executed Python experiments, historical repository records, and unexecuted Wolfram/Mathics probes. It includes candidate repairs and an independent exact-rational reference for the real W_{-1} branch. No successful Wolfram or Mathics package execution is claimed for this review.

Executive assessment

The repository is substantially more than an alternative spelling of `Series`. Its useful distinction is an explicit calculus of real asymptotic germs: complete power–log blocks, retained source and branch information, transported remainder descriptors, selected exact cores, and separate numerical and interval-certificate operations. Native Wolfram Language remains broader in general symbolic asymptotics, complex and multivariate input, and integration, summation, differential-equation, and discrete-sequence interfaces. The sensible comparison is therefore between *contracts and representations*, not a contest based on function-name counts. The package already acknowledges this distinction through its separate native-result representation.[1, 2, 17, 24, 26, 29]

The earlier reviews are unusually extensive. The maintained register consolidates 167 finding entries from 27 reports, including 44 entries in the third wave. Repeating their assumption, logarithmic-tail, request-normalization, refinement, or native-storage findings would not add much. This report instead concentrates on the Mathics adapters and portable validation machinery introduced after the third-wave source snapshot, with targeted inspection of the shared code they consume. The maintained register and wave-three crosswalk are the exclusion baseline; they are not treated as proof that every historical issue is still present.[4, 5]

ID	Classification	Finding and evidence boundary
R1	High; semantic portability	Exact and reverse ProductLog conversion are unsafe; a separate arity gate blocks numerical two-argument use. Source trace and independent backend witnesses; package probe unexecuted.
R2	High; validation integrity	A detected source change is forgotten after restoration; the copied suite is unchecked. Both false-green traces reproduced with the exact upstream Python runner.
R3	Medium; robustness	Nonfinite and huge timeouts reach process APIs and crash reporting. Reproduced for `nan`, `inf`, and `1e100`.
R4	Medium; CI coverage	Workflow triggers omit direct command dependencies, notably the standalone builder. Source-level gap and independent pattern checks.
R5	Performance risk	Sparse coefficient and Fourier-amplitude paths densify by degree; FirstPosition collects every match. Structural costs, not measured kernel benchmarks.
R6	Adapter mismatch	Omitted Limit direction becomes a left-hand limit rather than the native two-sided default. No current public wrong-result path is established.

The strongest immediately actionable changes are R2's monotone evidence state and checked suite snapshot, R3's finite bounded timeout policy, and R1's complete argument-conversion repair or a fail-closed boundary for unsupported branches. The supplied runner changes pass focused Python tests. The special-function repair is an independently tested candidate, not an accepted Mathics integration patch.

Execution boundary. There are 22 passing Python unittest methods in three runs: ten runner tests, nine independent semantic/mathematical tests, and three synthetic-source staging tests. These counts are *not* package-test counts. A Mathics installation could not be obtained in the execution environment, and the connected official Wolfram evaluator failed to connect. No whole-package suite, official-kernel differential run, Mathics integration run, or kernel performance benchmark was completed. The archive preserves the actual successful checks and supplies fresh-process kernel characterizations for the remaining obligations.

Contents

1 Scope, revision, and evidence discipline

1.1 The reviewed object is a fixed revision

All package-source conclusions refer to

`7d1bc832895cc90a9b2a978b7b7684acab908bd2`

not an evolving `main` branch. The package is named `AsymptoticAnalysis`; `AsymptoticInverse` remains a public function. Source modules live under `src/Kernel`; the repository-root `AsymptoticAnalysis.wl` is a generated standalone distribution. This distinction matters when applying a repair: editing only the generated file is not a maintained source change.[1, 2]

The latest third-wave reports examine revision `6687962f3c85`. The reviewed revision is 28 commits ahead of that baseline. The comparison identifies all fifteen `Mathics*.wl` adapter modules, the portable runner, and the Mathics workflow as additions after it. This gives a concrete incremental review surface rather than an arbitrary choice to revisit old code. The report does not infer that every new line is defective or that no older recommendation could be relevant to a new mechanism.[6, 5]

The source review covered all fifteen Mathics adapter modules, their entry-point loading and public declarations, the complete portable Python runner and workflow, and selected shared consumers: exact-core inversion, Lambert construction, source-coordinate reconstruction, forward arithmetic, Fourier coefficient normalization, and native dispatch. The broad feature comparison uses the maintained guide and current primary documentation. It is not a claim to have independently proved every theorem in the mathematical article or inspected every line of every shared engine.

1.2 How prior findings were excluded

The principal exclusion documents were the maintained complete finding register and the third-wave intake. Their crosswalks cover named findings and record unnumbered proposals. The review did not reread all 27 prior PDFs line by line. Instead, each proposed finding was checked against the consolidated mechanism and the source version to which the old observation applied. This is a meaningful but explicitly bounded novelty screen.[4, 5]

For example, the following are *not* new findings here: configured backend defaults, option-key equivalence, an overbroad search for `Function`, eager `Normal` of native output, generic observable Taylor ingress, exact-derived-value refinement, symbolic logarithm identities without real exponents, and the Fourier homogeneous zero-coefficient budget ordering. Nor does this report recommend implementing binary powering, Newton doubling, sparse product pruning, or a Mathics workflow as though they did not already exist.[4, 5, 19]

R5 deliberately touches the earlier performance work items P06/P08. Its additional content is the newly introduced dense `CoefficientRules` adapter, a concrete degree-versus-support workload through Fourier amplitudes, and the limitation of a commonly suggested `Position` optimization on the actual Mathics version. These are specified as deltas, not relabeled discoveries of a general resource-budget problem.

1.3 Four distinct kinds of evidence

Source-established mechanism means that a control flow, conversion rule, allocation pattern, or missing guard follows from the inspected implementation. It does not establish every detail of

host evaluation or public reachability.

Executed independent check means that code in this archive ran successfully in Python. The runner experiments execute the exact upstream runner, but their child processes are intentionally synthetic protocol producers, not Wolfram kernels. The mathematical experiments use SymPy 1.14.0, mpmath 1.3.0, and exact rational arithmetic. They can falsify an argument mapping or verify a mathematical enclosure without proving package integration.

Historical repository evidence means a previously recorded run in the repository. It remains attached to its recorded source and kernel. Neither a receipt nor a large passing count is silently promoted into a new acceptance run of the reviewed revision.

Unexecuted characterization is a concrete script or call supplied to establish the remaining runtime behavior. Such calls are marked as unexecuted throughout. A predicted branch failure is not printed as if it were an observed console transcript.

This discipline is particularly important for compatibility work. A wrapper may preserve a formal expression while its eventual numerical evaluation is wrong; a native control may preserve definitions without proving all numerical behavior; and a test harness may print correct per-case values while invalidating its own source-provenance claim.

2 Architecture and mathematical contracts

2.1 The ordinary germ representation

A useful model for the package's ordinary inverse engine is

$$f(x(u)) = y_0 + a u^p \left(1 + \sum_{j=1}^{m} u^{\delta_j} B_j(\log u)\right), \qquad u \to 0^+, \tag{1}$$

where the local chart is $x(u) = x_0 + \sigma u$ at a finite endpoint or $x(u) = \sigma/u$ at an infinite one. The leading coefficient and branch must satisfy the appropriate real-domain hypotheses, the perturbation gaps are positive, and each B_j is a polynomial. The ordinary numerical-weight path requires exact admitted real exponents; a separately restricted depth path accommodates symbolic exponents.[2, 14]

The central unit is a *complete block* at one weight, not one monomial after expanding a logarithmic polynomial. A remainder descriptor

$$\texttt{PowerLogRemainder}[u, \beta, k] \quad \rightsquigarrow \quad O\left(u^{\beta}(1 + |\log u|)^k\right) \tag{2}$$

retains the logarithmic degree that a bare power-order notation can hide. This is qualitative asymptotic information: it does not contain a computed constant or threshold. The package's interval-certificate API is a separate operation with stronger inputs.[2]

The code uses sparse rows, precision pairs, and explicit combination rules. A multiplication must account for retained-term times unknown-tail contributions as well as omitted finite products. A nonlinear operation must preserve the tail inherited from its argument even when more output precision is requested. These are not new recommendations: they explain why compatibility helpers used beneath this machinery are part of the mathematical trusted base rather than cosmetic syntax adapters.

2.2 There is more than one scale and more than one meaning of order

The implementation contains ordinary power–log germs, reciprocal and iterated logarithmic scales, retained Lambert and other exact cores, Gamma and Barnes inverse scales, flat exponential sectors, finite Fourier-polynomial coefficients, and composite envelopes. Shared arithmetic cannot infer identical closure properties across them. In particular, an exact prefactor can carry most of an expression's growth while the recorded cutoff applies only to its correction bracket.[1, 2, 3]

A Fourier block has the form

$$u^{\alpha} \sum_{\omega \in \Omega} P_{\omega}(\log u) e^{i\omega \log u}, \tag{3}$$

with finitely many exact real frequencies and conjugacy conditions for a real expression. The source weight, number of frequencies, and polynomial degree of the amplitude are different dimensions of work. R5 arises precisely because bounding one of these dimensions does not bound another.[16]

Exact-core marker expansions add another distinction. A coefficient of a perturbation marker is not necessarily a single exponent-sorted block. The exact core inverse may remain in every coefficient and in the remainder coordinate. Consequently, a host evaluator's handling of that exact core can affect both ordinary numerical use and later transformations that trust the core identity.[14, 15]

2.3 Native results are intentionally a different contract

The held `AsymptoticExpansion` entry point can choose the package engines or explicitly delegate to `System‘Series` or `System‘Asymptotic`. Automatic routing retains successful package results and searches compatible native alternatives for admitted native routes and selected representation failures. Explicit package-only constraints are not meant to disappear merely to obtain a native answer.[17]

The `Kind -> "Native"` representation preserves native output and request metadata without pretending that a formal native order establishes the package's analytic remainder theorem. Analytic operations can refuse such an object. That separation is a sound architectural choice. The fact that native `Normal` may still contain an infinite expression is not, by itself, an error: normalization is not synonymous with evaluation of all symbolic sums.[17, 1]

The portability consequence is subtle. Native delegation in Mathics delegates to *Mathics's implementation*, not to a remote official Wolfram kernel. The creation of an inert `System‘Asymptotic` name enables consistent symbol resolution; it does not create an asymptotic backend. Native-result metadata should therefore be read together with the actual runtime and outcome, never just the requested backend's name.[7, 9]

3 Comparison with built-in Wolfram Language

3.1 Capability comparison

The native system should not be described as supporting only ordinary Taylor series. Its documented `Series` interface includes Laurent, Puiseux, logarithmic, multivariate, and complex examples, while `Asymptotic` has a broader asymptotic interface and can use non-power scales. The package's distinction is the specific sparse representation and retained operational contract for its admitted families.[24, 26]

Task	Built-in Wolfram Language	AsymptoticAnalysis at the reviewed revision
Local forward expansion	`Series`, `Asymptotic`; broad symbolic-function support, native assumptions and direction options.	Explicit real power–log blocks and selected extended scales; native delegation for separately represented results.
Series representation	`SeriesData` stores coefficients on a rational exponent lattice, including an order endpoint.	Sparse admitted exact weights and polynomial logarithmic blocks; optional native projection only when representable.
Local inversion	`InverseSeries` for appropriate series data; `AsymptoticSolve` for asymptotic solution branches.	Real source charts, branch metadata, generalized inversion methods, selected exact cores, and inverse residual operations.
Multivariate and complex problems	Native interfaces include multivariate series and real/complex asymptotic solution problems.	Custom engines are predominantly one-variable real germs. Native results do not imply custom complex-sector closure.
Special functions	Extensive direct expansion and asymptotic functionality.	Targeted Gamma, Barnes, Lambert, Fourier, flat, and Dirichlet-related constructions with explicit scale conventions.
Discrete sequences	`DiscreteAsymptotic` supports integer-limit sequences and expressions involving sums, products, generating-function coefficients, and recurrence solutions.	No equivalent dedicated public discrete backend appears in the inspected dispatcher. This is a scope comparison, not a newly discovered defect.
Integration and summation	Dedicated `AsymptoticIntegrate` and `AsymptoticSum`, among related interfaces.	Existing germ operations are not a replacement for those general solvers. The earlier adapter and integration proposals remain prior work.
Differential equations	`AsymptoticDSolveValue` and related native facilities.	No general differential-equation solver is claimed by the custom calculus.
Error information	Native orders and asymptotic output have their own semantics; symbolic coefficients need not constitute a user-supplied regularity proof.	Remainder descriptors, derivative-contract restrictions, provenance, and separate quantitative certificates for supported inputs.
Runtime reach	Official Wolfram evaluator and its numerical/symbolic libraries.	Official evaluator plus a substantial but bounded Mathics adaptation layer; equivalence is not established across all inputs.

The native entries are grounded in the respective reference pages; the package entries are grounded in the entry declarations, kernel source map, and native dispatcher.[24, 25, 27, 28, 29, 30, 2, 3, 17]

The phrase “input superset” must be handled carefully. It can mean syntactic acceptance, lossless storage of a native result, successful computation of the same value, or promotion into the custom analytic calculus. These are four different targets. A wrapper that preserves an unresolved native call has not computed its expansion. Conversely, a deliberate refusal to manufacture a remainder theorem can be the correct behavior even when the native system returns a formal series.

3.2 Equal numerical order arguments need not request equal information

For an ordinary expansion at zero, the package documents an exclusive cutoff. Thus the mathematical target of

```
AsymptoticExpansion[Exp[x], {x, 0, 3}, "Backend" -> "Package"]
```

is the finite part $1 + x + x^2/2$ with a remainder of order x^3. Native

```
Series[Exp[x], {x, 0, 3}]
```

retains the cubic term as well and has a fourth-order endpoint. These are explanatory consequences of the documented conventions, not outputs observed in this session. An explicit native backend intentionally uses native order semantics.[2, 24]

Benchmarking these calls with the same final integer is therefore not a matched-accuracy experiment. Neither is comparing a Gamma inverse bracket cutoff to an absolute power cutoff in the original target variable. A meaningful benchmark records the coordinate, exact prefactor, retained blocks, achieved remainder pair, branch assumptions, and whether optional native projection or display was included in the measured interval.

The existing reviews already propose a common request model and benchmark catalog. The new contribution here is narrower: a Mathics adapter can perform degree-sized or match-count-sized work *before* an otherwise sensible package budget is consulted. Instrumenting those dimensions is necessary even after request normalization has been completed.

3.3 What the package does particularly well

The code consistently tries to distinguish the finite approximation from its remainder and the original function from a finite model. It contains explicit conservative refusals, keeps source assumptions for later operations, and separates numerical diagnostics from interval proof. The repaired native export and assumption paths in the register show an effort to preserve information rather than obtain a visually attractive answer at any cost. These are strengths, not reasons to ignore the remaining proof obligations.[4, 2]

The exact-core architecture also makes a good compatibility strategy possible. A narrow, trustworthy provider for a difficult core can support several higher-level constructions. R1 shows the danger of depending on an unverified provider; Section 11 demonstrates a small reference implementation that checks a specific real branch independently of either symbolic evaluator.

4 R1: the exact ProductLog conversion boundary

4.1 A known limitation needs a sharper explanation

The repository already records a Mathics two-argument `ProductLog` problem. Its workaround normalizes the principal branch to the one-argument spelling, and its compatibility guide says that numerical nonprincipal Lambert evaluation remains unsupported. That warning is not a new finding. The additional issue is that the tagged Mathics implementation has a broken *exact conversion contract*, together with a separate numerical arity limitation. Describing all failures as a numerical mpmath argument swap is incomplete.[7, 10, 20, 21]

The relevant conventions are

$$\text{Wolfram:} \quad \texttt{ProductLog}[k, z] = W_k(z), \tag{4}$$

$$\text{SymPy:} \quad \texttt{LambertW}(z, k), \tag{5}$$

$$\text{mpmath:} \quad \texttt{lambertw}(z, k). \tag{6}$$

Both backend libraries put the argument before the branch. The two-argument Wolfram form puts the branch first.[33, 34]

Mathics 10.0.1's `ProductLog` class declares the SymPy and mpmath function names but does not implement an argument permutation. Its inherited `prepare_sympy` returns the elements unchanged; the inherited conversion passes those elements to the SymPy function. The inherited reverse conversion also rebuilds a Wolfram expression in the received order. The general conversion layer delegates a returned SymPy function to that reverse method. These are independently relevant directions of the bridge.[20, 21, 22]

There is a second important detail. The inherited `MPMathFunction` class has `nargs = {1}`. Exact arguments are handled through SymPy before the numerical arity check, whereas an inexact two-argument request does not pass the inherited mpmath arity gate. Consequently, a direct two-argument inexact request may remain unresolved instead of literally invoking mpmath with reversed arguments. A correct repair must address the exact conversion, reverse conversion, and numerical arity/order together.[21]

4.2 Independent witnesses

The following results were executed in SymPy 1.14.0, *not* in Mathics:

```
from sympy import E, LambertW, Rational
LambertW(0, E)                  # -oo
LambertW(E, 0)                  # 1
LambertW(-1, -Rational(1, 100))
LambertW(-Rational(1, 100), -1)
```

The first pair shows the consequence of sending the exact Wolfram argument pair `[0,E]` without permutation. The correct value is $W_0(e) = 1$, while the incorrectly ordered SymPy call evaluates to negative infinity.

For the minus-one branch, the correctly ordered call gives

$$W_{-1}(-0.01) = -6.4727751243940046947410578927244880371 04\ldots. \tag{7}$$

The incorrectly ordered SymPy expression `LambertW(-1,-1/100)` evaluates numerically to

$$\begin{aligned} &-0.31813150520476413531265425158766451720 35 \\ &+1.33723570143068940890116214319371061253 9\,i. \end{aligned} \tag{8}$$

The second parameter in that incorrect backend call is not an integer branch index. Its numerical behavior is a particularly clear witness that the conversion no longer represents the requested real function; no physical or mathematical interpretation should be assigned to that accidental branch parameter.

These experiments establish a backend counterexample to the source's argument mapping. They do not establish exactly which Mathics message, simplification, recursion, or refusal will occur for every public call. The archive includes separate exact and inexact fresh-process probes to resolve that integration behavior.

4.3 Why this affects the package's exact-core design

The current package helper has the following shape:

```
mathicsCoreProductLog[0, argument_] := System‘ProductLog[argument];
mathicsCoreProductLog[branch_, argument_] :=
  System‘ProductLog[branch, argument];
```

It rewrites the `ProductLog` construction sites in four private consumers, including Lambert construction and automatic exact-core inversion. The principal-branch case avoids the two-argument representation. The nonprincipal case forwards it unchanged. This is a sensible limited workaround for branch zero, but it does not establish a safe nonprincipal symbolic carrier.[10]

Consider the core $f(x) = -x \log x$ on the small positive branch $x \to 0^+$. Set $v = -\log x > 1$. Then

$$y = ve^{-v}, \qquad v = -W_{-1}(-y), \qquad x = -\frac{y}{W_{-1}(-y)}. \tag{9}$$

The choice of branch is not optional: $W_0(-y)$ yields the other real solution, which approaches one rather than zero. The exact-core construction recognizes an affine-log-power core and chooses the minus-one branch in this regime. Its zero-perturbation case can retain the inverse as an exact expression with zero remainder.[14]

A targeted *unexecuted* package characterization is

```
s = AsymptoticCoreInverse[-x Log[x], 0, {x, 0}, {y, 0}];
N[Normal[s] /. y -> 1/100, 30]
```

The mathematically correct small inverse at $y = 1/100$ is

$$x = 0.00154493239882734560172624674509\ldots. \tag{10}$$

Substituting the incorrectly mapped backend value into the same algebraic formula gives

$$0.00168376379087222910557029040196 + 0.00707754188784727616472845893076\,i. \tag{11}$$

The latter calculation was performed independently in Python. It is not presented as a Mathics output. The source-level concern is that an exact expression used as a real inverse core depends on the unsafe host conversion; the public characterization must still determine whether that dependence manifests as a wrong value, an unresolved expression, a diagnostic, or an earlier refusal.

This distinction also matters for `TargetDomain`. A retained domain predicate involving the same faulty host special function is not an independent verification of that function's semantics. A domain check and an evaluation that share the same incorrect provider can agree with one another for the wrong reason.

4.4 Repair the entire bridge, not one observed symptom

A complete conversion repair has four obligations:

1. Convert the Wolfram two-argument pair (k, z) to the backend pair (z, k) before symbolic processing.
2. Convert a returned two-argument SymPy Lambert expression back from (z, k) to (k, z).
3. Admit both supported numerical arities and reorder the two-argument numerical call without bypassing the existing precision machinery.
4. Preserve the one-argument principal convention and test exact and inexact inputs independently.

The supplied `code/productlog_bridge.py` implements these method-level changes as an independently testable mixin. `code/stage_productlog_patch.py` stages equivalent methods into a separate candidate source file, refuses unexpected existing methods, and checks the resulting Python syntax. It never modifies the installed interpreter automatically.

The bridge methods are tested against a synthetic base class and real SymPy/mpmath operations. Three additional staging tests check method placement, preservation of the surrounding synthetic module, and refusal to overwrite a changed implementation. These are useful checks, but they do not replace integration tests against Mathics's expression classes, precision handling, evaluator recursion, conversion registration, and invalid-argument policy.

In particular, adding `nargs = {1,2}` is not a full `ProductLog` implementation. Integer-branch validation, derivative rules, branch-cut limits, and every special value remain separate obligations. The candidate is intentionally narrower than a claim to complete special-function compatibility.

A package-local alternative is to retain a nonprincipal core in an inert package-owned representation until a verified provider is available, or to refuse the unsupported operation before promising a usable exact core. Merely relabeling the metadata as "nonprincipal numerics unsupported" is insufficient when a later exact simplification can enter the same unsafe conversion path. A scoped inert representation would need explicit rules for substitution, display, differentiation, comparison, and numerical evaluation; otherwise it simply moves the ambiguity.

4.5 Acceptance criteria

The first integration gate should characterize `ProductLog[0,E]`, `ProductLog[-1,-1/100]`, and an inexact two-argument call on a clean Mathics 10.0.1 installation. It should then exercise the package helper and the exact-core public call separately. Branch zero, branch minus one, and at least one genuinely complex branch should be tested through both conversion directions.

A real-branch assertion must check more than the residual $we^w = z$. On $-1/e < z < 0$, both real branches satisfy that equation. For the minus-one branch, the test also needs $w < -1$, and for the small inverse it needs $0 < x < 1/e$. Section 11 supplies a rational enclosure oracle that makes this branch check independent of Mathics, SymPy, and mpmath.

5 R2: acceptance evidence must be monotone

5.1 A detected mutation can disappear from the final record

The new portable runner does many things correctly. It selects unique test IDs, rejects unmatched selections and zero selected cases, launches a fresh process per case, validates the output protocol, distinguishes process failures and timeouts, and compares source fingerprints. It also copies

the test suite to avoid changes to a streamed input file during evaluation. This is a considerably better foundation than counting printed "pass" strings.[18]

The defect lies in the lifetime of its evidence. Each call to `write_report` computes a new `after = fingerprints(source)` and records

```
"SourcesUnchangedDuringRun": before == after
```

The final exit status tests the final report's value. The comparison is not accumulated across checkpoints. A source change observed after one case can therefore be forgotten after a later case restores the original bytes.[18]

The reproduced trace is:

Step	Source/test action	Recorded behavior
0	Begin with source bytes A.	Baseline fingerprints represent A.
1	First synthetic case changes source to B.	The runner checkpoint records `SourcesUnchangedDuringRun = False`.
2	Second synthetic case restores source A.	Both cases have valid success protocol records.
3	Final report compares current A to initial A.	It records `True`; the exact upstream runner exits with code 0.

This is stronger than the familiar observation that an edit and restoration between two checks can be invisible. Here the runner *saw and recorded* the invalidating event. Its final state then erased that knowledge.

The experiment executes the complete upstream Python runner preserved in `fixtures/run_mathics_tests_upstream.py`. The fixture was verified byte-for-byte against the fetched repository blob. The child executable is a bounded synthetic process implementing the runner's documented output protocol. Its kernel identity explicitly says that it is a Python fake protocol process, not a Wolfram or Mathics kernel. The experiment tests the runner's provenance logic, not mathematical package behavior.

The controls matter. An unchanged-source success case exits zero. A persistent source change exits nonzero. Malformed protocol records are rejected. Thus the reproduced false green is not caused by replacing the runner's parser with a permissive imitation or by disabling all integrity checks.

5.2 The copied test suite is not actually immutable

The runner copies `MathicsTests.wl` into a temporary directory and invokes the copy. Its fingerprint function includes the *original* suite and source files, not the copied suite. The copy is never checked before or after individual launches. Naming the copy "frozen" describes the intent, not an enforced invariant.[18]

A second synthetic trace changes that copied suite during the first case. The second case then observes the changed bytes and returns the marker `MUTATED SUITE`. The upstream runner still records two successes, source equality, and exit code zero. The supplied test does not assert that an adversarial Wolfram program was run; it proves the narrower fact that the runner can accept a batch whose later cases received a different suite file from the one it recorded.

This is a provenance defect, not a claim that every repository acceptance receipt is invalid. The

separate native definition-comparison tool has its own snapshot and script-validation logic. Its behavior must not be inferred from the portable runner's implementation. Likewise, an ordinary read-only CI checkout makes some mutations less likely without making the source-equality statement logically stronger.[7]

5.3 A minimal tested repair

The supplied runner patch makes observed source inequality sticky:

```
sources_unchanged = True

def write_report(complete):
    nonlocal sources_unchanged
    after = fingerprints(source)
    sources_unchanged = sources_unchanged and before == after
    # Store sources_unchanged, not only the latest comparison.
```

It also checks the copied suite immediately before each case and after the process returns, retains a sticky `SnapshotMatchedAtCheckpoints` flag, stops launching further cases after a mismatch, and includes that flag in the final exit gate. Its completion flag reflects whether all selected cases were executed, rather than simply whether the outer loop reached its cleanup code.

The patch is intentionally staged into a separate destination and uses strict single-occurrence source contexts. Applying it to an unexpected revision fails instead of guessing a replacement location. The test suite covers the repaired source-change trace, repaired copied-suite trace, unchanged success, and timeout validation described in R3.

Scenario	Upstream result	Candidate result
Unchanged source and valid records	Exit 0; two successes.	Exit 0; two successes.
Detected source change, then restoration	Exit 0; final equality flag true.	Exit 1; equality flag remains false.
Copied suite changed in first case	Exit 0; second case reads changed suite.	Exit 1; second case is not launched; snapshot flag false.
Persistent source change	Exit 1.	Preserved by sticky logic.
Malformed protocol control	Exit 1.	The candidate does not alter protocol parsing.

Only the explicitly executed candidate cases are counted as candidate tests; the persistent-change and malformed-protocol rows describe preserved source behavior and upstream controls, not additional candidate executions.

5.4 What the repair still does not prove

Checkpoint validation is not a proof that bytes never changed between checkpoints. Nor is a process with write access to its own source an adversarially isolated execution environment. The candidate does not claim either property. A stronger practical design would copy the selected entry point and its complete module dependency tree, run exclusively from that snapshot, verify the copied inputs at each launch and completion, and record the distinction between original-source stability and executed-snapshot equality.

Even that design should name its evidence accurately. “Matched at all observed checkpoints” is narrower and more truthful than an unconditional statement that no edits occurred. Source mutation, launch failure, protocol failure, test failure, and incomplete execution are separate state components. Each invalidating component should evolve monotonically over a batch:

$$I_{n+1} = I_n \vee \text{newInvalidity}_{n+1}\,. \tag{12}$$

A final report can summarize this state, but must not reconstruct it solely from the final filesystem or the last successful process.

This proposal is a concrete repair to the new runner, not a repetition of the older general recommendation to add CI. The Mathics workflow already exists; the problem is the meaning of one of the acceptance facts it consumes.

6 R3: validate the timeout domain before process creation

The portable runner accepts `-timeout` using Python’s floating-point parser and checks only whether the value is nonpositive. This admits both NaN and positive infinity. It also admits finite values far outside a platform process API’s representable timeout range.[18]

The exact upstream runner was exercised on Python 3.13.5/Linux with a bounded sleeping synthetic child. The results were:

Argument	Observed exception	Report state
`nan`	`ValueError`: NaN cannot be converted to an integer in timeout processing.	`RunComplete = False`; zero executed cases; selected cases remain unrun.
`inf`	`OverflowError`: infinity cannot be converted to an integer.	Same incomplete-report pattern.
`1e100`	`OverflowError`: timestamp too large for the process timing representation.	Same incomplete-report pattern.

These are not false-success results: the runner exits nonzero. The defect is an invalid configuration reaching process creation and internal timing machinery, producing an exception instead of an ordinary argument diagnostic. The exact exception text and platform limit can vary with Python and operating system; the missing finite/range validation is source-established independently of that variation.

The runner’s default JSON writer also permits nonfinite floating values. Without earlier validation, the timeout field can contain a nonstandard numeric token in a partial report. This is a source-level serialization consequence, not a claim that every downstream consumer was tested or failed.

The supplied candidate chooses an explicit supported policy:

```
if not math.isfinite(args.timeout) or not 0 < args.timeout <= 86400:
    parser.error("--timeout must be finite and in (0, 86400] seconds")
```

The one-day ceiling is an engineering policy, not a mathematical necessity or a claim about the maximum representable timeout on every platform. It is deliberately generous compared with

the workflow's five-minute per-case budget. A project can choose a different documented cap, but should validate it before launching any process.

The candidate tests reject NaN, infinity, `1e100`, zero, and a negative value through `argparse`, with exit code 2 and no child launch. The ordinary success path remains unchanged. Input-domain validation is a better repair than catching an arbitrary low-level overflow and labeling it a kernel failure: the kernel did not cause the invalid configuration.

7 R4: workflow triggers should cover the commands they run

The repository has a real Mathics CI workflow. It runs a matrix of modular and standalone entry points against five suite groups, installs the pinned Python dependencies, checks standalone generation, runs the Python Mathics-related tests, and runs portable exact regressions. It has job and process timeouts and uploads reports even on failure. A review that recommends adding a Mathics workflow from scratch would be out of date.[19]

The path trigger, however, includes only the kernel tree, the standalone file, lowercase `mathics`-matching validation files, the explicit `MathicsTests.wl` file, and the workflow itself. One of the job's commands is

```
python validation/build_standalone.py --check
```

but a change confined to `validation/build_standalone.py` does not match those trigger paths. The standalone builder is part of what the job executes, so this is a direct dependency omission. The capitalized `validation/CheckMathicsDefinitions.wl` is another unmatched validation asset; case sensitivity prevents the lowercase wildcard from covering it.[19]

Independent path-pattern checks in the archive record:

Path	Matches the relevant recorded patterns
`validation/run_mathics_tests.py`	Yes
`validation/MathicsTests.wl`	Yes
`validation/build_standalone.py`	No
`validation/CheckMathicsDefinitions.wl`	No

The decisive claim does not depend on reproducing GitHub Actions scheduling in a local simulator: the builder's name plainly matches neither the explicit entries nor the lowercase wildcard. The local checks are a compact regression oracle for that dependency list.

A broad `validation/**` trigger is a simple conservative repair. An explicit list can avoid unrelated jobs, but should be generated from or tested against the actual command dependencies. The project should decide whether a builder-only change is intentionally excluded; absent such a policy, executing a file without triggering on its edits creates an avoidable blind spot.

A useful related acceptance test would ensure that the union of configured suite groups equals the groups declared by the portable suite and that the intended partition has no accidental omissions. This is a development recommendation, not a claim that a currently declared group is missing. The distinction keeps a concrete present defect separate from a preventive test.

8 R5: sparse series can enter dense compatibility paths

8.1 The new CoefficientRules shim is dense by degree

The Mathics algebra adapter implements the univariate `CoefficientRules` case by expanding the polynomial, obtaining `CoefficientList`, mapping indices to powers, removing zero coefficients, and reversing the result. For ordinary inputs this is a straightforward semantic fallback. Its cost, however, is governed by the *largest degree*, not the nonzero support of the requested output.[11]

For

$$P_N(\ell) = 1 + \ell^N, \tag{13}$$

the sparse result contains two rules, but `CoefficientList` must represent $N + 1$ coefficient positions. Filtering zero entries afterward cannot recover the memory and construction work already spent. The independent checks produce the following structural counts:

N	Nonzero support	Dense coefficient positions
100	2	101
10 000	2	10 001
100 000	2	100 001

These are exact representation counts, not measured Mathics timings, resident-memory figures, or evidence that a particular large public request was executed. The asymptotic distinction is nevertheless real: this adapter changes a support-sized representation task into a degree-sized intermediate task.

The earlier native `SeriesData` export work does not settle this issue. That work guards a different dense representation boundary with its own rational-lattice and index-range constraints. A sparse native-export refusal does not prevent a later coefficient helper from allocating a dense polynomial array. The newly introduced adapter is an additional site that needs its own resource policy.[4, 11]

8.2 A concrete Fourier-amplitude workload

There is also a direct shared consumer whose cost is not captured by the frequency budget. `fourierPoly` normalizes an amplitude through `CoefficientList`; `fourierEnvelope` enumerates coefficient positions to build a magnitude envelope. The public Fourier inverse engine has source-pair and frequency limits, but those limits do not bound the degree of a polynomial amplitude.[16]

A candidate workload is

```
AsymptoticFourierInverse[
  x + x^2 (1 + Log[x]^N), {x, 0}, {y, 3},
  "MaxTerms" -> 10, "MaxFrequencies" -> 1]
```

with a concrete positive integer substituted for `N`. It has one Fourier frequency, zero, and a sparse logarithmic amplitude. A small value such as 20 is supplied in the unexecuted probe driver. Starting with an enormous degree is neither necessary nor a useful first test.

The mathematical inverse coefficients make the workload more informative. Write $B(L) = 1 + L^N$ and seek

$$x = y - y^2 B(\log y) + y^3 C(\log y) + \cdots . \tag{14}$$

Expanding $x^2 B(\log x)$ through order y^3 gives

$$C(L) = 2B(L)^2 + B(L)B'(L). \tag{15}$$

Thus, for fixed $N \geq 1$, the first omitted complete block after the displayed second-order approximation has logarithmic degree $2N$. The expansion

$$x = y - y^2(1 + \log^N y) + O\big(y^3(1 + |\log y|)^{2N}\big) \tag{16}$$

needs only a few sparse polynomial monomials at these low orders, even though a degree-indexed intermediate is large. This is an independently derived test family, not an observed package result.

The appropriate resource dimensions are consequently at least source-weight support, frequency support, amplitude support, maximum amplitude degree, and coefficient expression cost. These need not become five user-visible knobs immediately. An internal degree admission guard and sparse amplitude representation can provide a bounded first repair while the public policy is decided.

8.3 A bounded candidate, and its limitations

The archive includes `code/MathicsSparseHelpers.wl`, which collects an already expanded univariate polynomial by the exponents of its nonzero summands and then combines equal exponents. It lives in an independent audit context and does not replace any system symbol or install itself into the package. Its expected output for $1 + \ell^N$ is

```
{{N} -> 1, {0} -> 1}
```

without an explicit zero array of length $N + 1$.

This is an unexecuted Wolfram-language candidate. Moreover, it does not make `Expand` free: expanding $(1 + \ell)^N$ genuinely produces $N + 1$ nonzero monomials. Nor does it solve coefficient-expression explosion or arbitrary semantic zero detection. The claim is only that a polynomial already sparse in its expanded support need not be densified merely to return sparse coefficient rules.

For Fourier amplitudes, an implementation should preserve exact coefficient grouping, conjugacy checks, and the correct logarithmic degree of omitted blocks. A fast implementation that drops an unobserved coefficient or lowers a remainder degree would be worse than the slow current one. Regression tests should compare sparse and dense methods on small degrees, cancellations, symbolic real coefficients, zero polynomials, and algebraic coefficients before exercising the high-degree sparse family.

8.4 FirstPosition builds the entire match list

The compatibility implementation of `FirstPosition` calls `Position` and then takes the first result. Its lazy missing-value behavior and support for patterns address actual interpreter gaps, but the helper materializes every matching position even when the first match is sufficient. An expression with M matches can therefore produce an M-element result list before returning one position.[9]

This is a performance observation, not a demonstrated wrong first-position result. The official documentation describes depth-first behavior and a heads policy; a replacement must preserve those semantics and the lazy default. Changing the traversal order to make early exit easier is not a correct optimization.[31]

A tempting patch is to add a maximum-result argument to `Position`. That is not a drop-in fix for the inspected interpreter: Mathics 10.0.1's `Position` implementation accepts the expression, pattern, optional level specification, and options, and its callback appends all matches. It does not implement the extra result-count form needed by that suggestion.[23]

A bounded walker or upstream early-stop callback is therefore the concrete next step. Its acceptance tests need root and head matches, nested expressions, explicit positive and negative level specifications supported by the host, absent matches, and an effectful missing default. For performance measurement, count visited nodes and collected matches separately. The source proves unnecessary result-list materialization; it does not quantify the improvement for a representative package workload.

9 R6: an implicit left-hand Limit is not native default parity

The Mathics simplification module supplies a private `Limit` wrapper. Its option default is `Automatic`; its direction translation maps `"FromAbove"` to -1, `"FromBelow"` to $+1$, and `Automatic` to $+1$. The first two mappings have the intended one-sided meaning. The final mapping makes an omitted wrapper direction a left-hand limit.[12]

The current Wolfram documentation instead lists the default direction as `Reals`, the two-sided real approach. Therefore the wrapper's omitted-option behavior is not the same contract as the native default.[32]

The elementary separating example is

$$g(t) = \frac{|t|}{t}, \qquad \lim_{t \to 0^-} g(t) = -1, \qquad \lim_{t \to 0^+} g(t) = 1. \tag{17}$$

The two-sided limit does not exist. The left/right values were also checked independently in SymPy. The archive's unexecuted kernel probe asks the private wrapper and the native control for the omitted, from-above, and from-below cases separately.

This is not presented as a confirmed public package bug. The package's internal local coordinate is positive, and many consumers explicitly request the intended side. No current public entry point was demonstrated in this review to rely on this wrapper's omitted direction for a boundary-sensitive expression. It would be misleading to turn an adapter contract mismatch into a claim of a reproduced wrong expansion.

There are two coherent repair policies. One is to make internal one-sided requests mandatory and name the helper accordingly, so it no longer appears to be an unrestricted native `Limit` replacement. The other is to support a genuine default two-sided operation by evaluating the two sides and accepting a result only when the relevant equality and existence conditions are established. In either policy, unresolved assumptions must remain unresolved; simply choosing whichever side evaluates first is not a proof of the two-sided limit.

The distinction is especially relevant to the package's inverse charts. The meaningful direction is the direction in the *current local coordinate*, not necessarily the original source variable's direction string. Tests should therefore include a finite approach from below and a negative-infinity source chart, but only after identifying the actual consumers that omit a direction. This is a targeted adapter audit, not a repetition of the third-wave native Taylor endpoint finding.

10 Mathics compatibility: a contract-level assessment

10.1 What is genuinely implemented

The port is substantial. It is not simply a successful `Get` followed by a few arithmetic examples. The source supplies evaluator-specific behavior for explicit module returns, lazy association lookups, held extraction, polynomial and sign reasoning, Taylor providers, selected Stirling expansions, Fourier helpers, numerical-core construction, list-cache avoidance, retained refinement, certificate history updates, and arithmetic formatting. It installs the compatibility names for package parsing while avoiding replacement of Mathics's global system-function definitions.[7, 9, 3]

The package also handles practical evaluator differences: streamed parsing after loading, an explicit larger Mathics iteration budget in its test environment, conservative fallback when domain proofs fail, and process isolation in the portable runner. It would be unfair to describe the port as wholly untested or to recommend the same elementary shims again.[7, 18]

At the same time, "complete compatibility is a goal" is not a passing acceptance statement. The guide explicitly says that the final consolidated Mathics run is pending and distinguishes focused examples from all parameter ranges. That wording is appropriately cautious. The new findings refine what should be checked next, rather than contradicting a nonexistent claim of full compatibility.[7]

10.2 Compatibility matrix

Boundary	Implemented strategy	What still needs to be established
Load, reload, namespaces	Mathics-only bootstrap; private definitions cleared for reload; compatibility context removed from the caller path.	Clean modular and standalone execution on the final source and supported interpreter versions. No new reload defect is asserted here.
Module and Return	Dynamically distinct catch/throw tags emulate the package's explicit module exits, including nested invocation.	All actually used held/control-flow forms, not arbitrary full-language equivalence.
Associations and held extraction	Bounded lookup/update/key operations and held traversal avoid known evaluator differences.	Side effects, defaults, list keys, and callable binding remain regression-sensitive.
Realness and sign reasoning	Exact consequences of explicit conjunctions; recursive finite-real and sign checks; unknown remains unknown.	Broader domains require proofs, not numeric samples. No newly established unsound sign rule is reported.
Taylor special functions	Defining-series adapters for selected hypergeometric and polylogarithm inputs; constant outer factor and vanishing monomial argument.	More argument forms and parameter ranges; preserve termination and analytic tails.

Boundary	Implemented strategy	What still needs to be established
Large positive LogGamma	Finite Stirling model evaluated through the existing jet algebra with an explicit omitted tail.	Full family/domain integration and resource behavior; no false exactness issue found in the inspected specialization.
Lambert cores	Principal branch normalized to one argument; nonprincipal forms retained.	R1's exact/reverse conversion and arity issue; real branch validation independent of the provider.
Fourier and coefficient algebra	Exact trigonometric rewrites, coefficient and take helpers.	R5's degree-versus-support allocation and matched performance checks.
Native delegation	Interpreter-native symbols and results remain separate from custom analytic contracts.	An inert symbol is not a supported backend; broader native functionality cannot be assumed.
Numerical certificates	Selected exact rational certificate examples and a local history-index adapter.	General numerical and special-function coverage is not established by those examples.
Display	Direct arithmetic upvalues and bounded output formatting.	Notebook/front-end behavior needs its own tests; `Normal` and `InputForm` are useful inspection paths, not proof of display parity.
Validation records	Fresh-process portable protocol and historical official-kernel preservation evidence.	R2/R3 repairs, trigger coverage, final immutable-source acceptance, and clearly separated runtime identities.

The matrix summarizes inspected adapter implementations and the guide's stated scope. It deliberately avoids a numerical compatibility percentage: the input space has no defined denominator, and a special function's existence does not establish its supported overloads or branches.[7, 9, 11, 12, 10]

10.3 Why preservation receipts cannot be combined into a larger claim

The repository records a historical official-Wolfram comparison with 1,452 passes and the same 12 pre-existing failures across 79 suites, relative to the untouched baseline. It separately records later definition comparisons as upstream changes were merged. The guide identifies the latest comparison's 2,051 modular and 2,050 standalone package symbols and selected system-definition/behavior probes. It explicitly does not relabel the earlier full-suite run as a full run of those later changes.[7, 8]

That is the correct distinction. Equality of definitions is evidence about the installed definitions under a particular evaluator and normalization; it is not a theorem of behavioral equivalence in all surrounding programs. A focused portable example is evidence about its own input and output; it is not a proof that every parameter range of the same family works. An official-kernel preservation result is not a Mathics test result.

This review's failed evaluator connection adds no negative mathematical evidence about the package. It is only a limitation of the review environment. The independent tests are deliberately designed to make progress despite that limitation: they exercise the actual Python runner,

inspect the tagged interpreter conversion source, and prove a specific real-branch enclosure with rational arithmetic.

10.4 A stronger semantic acceptance matrix

For each adapter, the useful unit is a *specific overload and contract*. For `ProductLog`, test one versus two arguments, exact versus inexact input, both conversion directions, branch zero versus minus one versus a complex branch, and symbolic retention versus numerical specialization. For `Lookup`, test present versus missing keys with an effectful default. For `Limit`, test explicit sides separately from omitted direction. For coefficient rules, test sparse high degree separately from genuinely dense support.

The same test should run against both modular and generated standalone entry points, with the runtime tuple recorded. A successful symbolic object should be followed by a small set of downstream operations that cross provider boundaries: substitution, simplification, numerical evaluation, and branch checks. This avoids the specific R1 failure mode in which apparently successful exact-core construction leaves a dangerous special-function expression for later use.

The matrix should distinguish success, deliberate unsupported behavior, unproved mathematical condition, resource exhaustion, unresolved host expression, host exception, and test-infrastructure failure. This is not a proposal to replace the already discussed public result schema; it is a concrete evidence vocabulary for the newly added evaluator adapters and their tests.

11 An independent rational certificate for the real minus-one branch

11.1 Why a branch-specific oracle is useful

The R1 regression should not use the same host conversion as both implementation and oracle. High-precision mpmath is a useful numerical control, but a small exact-rational reference can do more: it can certify an interval and the intended real branch without any floating-point logarithm or special-function evaluation.

The supplied `code/lambert_certificate.py` addresses the narrow problem

$$w = W_{-1}(-t), \qquad t \in \mathbb{Q}, \quad 0 < t < 1/e. \tag{18}$$

It returns rational endpoints for w and, by monotone transformation, the small inverse of $-x \log x = t$. This is not a complete Lambert-W library, not a replacement for arbitrary-precision complex special functions, and not an integrated extension of the package's certificate API. It is a reference artifact tailored to the new compatibility risk.

11.2 An exact logarithm enclosure

For $1 \le r \le 2$, set

$$s = \frac{r-1}{r+1}, \qquad 0 \le s \le \frac{1}{3}. \tag{19}$$

The elementary atanh identity gives

$$\log r = 2 \sum_{j=0}^{\infty} \frac{s^{2j+1}}{2j+1}. \tag{20}$$

After n terms, define

$$L_n(r) = 2\sum_{j=0}^{n-1} \frac{s^{2j+1}}{2j+1}, \tag{21}$$

$$U_n(r) = L_n(r) + \frac{2s^{2n+1}}{(2n+1)(1-s^2)}. \tag{22}$$

Every summand is nonnegative. Replacing every denominator in the tail by the smallest denominator, $2n+1$, and summing the geometric progression proves

$$L_n(r) \le \log r \le U_n(r). \tag{23}$$

For rational r, both endpoints are rational and can be computed exactly.

For a general positive rational q, choose an integer k with $q = 2^k r$ and $1 \le r < 2$. Then

$$\log q = k \log 2 + \log r. \tag{24}$$

The implementation bounds both logarithms using (22). When $k < 0$, multiplication by k reverses the interval endpoints; the code explicitly swaps the appropriate bounds. The power-of-two range reduction is performed with integer bit lengths and rational arithmetic, not an approximate logarithm.

This derivation supplies the reason that the bounds are valid. The tests against a 100-digit numerical oracle are additional implementation checks, not the proof of the enclosure theorem.

11.3 Reduce Lambert inversion to a monotone scalar equation

Write $v = -w > 1$. The equation $we^w = -t$ becomes

$$t = ve^{-v}. \tag{25}$$

Equivalently, define

$$h_t(v) = v - \log(v/t). \tag{26}$$

For $v > 1$,

$$h_t'(v) = 1 - \frac{1}{v} > 0. \tag{27}$$

Furthermore $h_t(1) = 1 + \log t < 0$ precisely when $t < 1/e$, and $h_t(v) \to +\infty$ as $v \to \infty$.

Proposition 11.1. *Let $t > 0$ be rational. Suppose rational numbers $1 \le a < b$ satisfy $h_t(a) < 0 < h_t(b)$. Then exactly one $v \in (a, b)$ solves $t = ve^{-v}$. Its negative is $W_{-1}(-t)$, so*

$$-b < W_{-1}(-t) < -a. \tag{28}$$

The small real inverse of $-x \log x = t$ lies in

$$\frac{t}{b} < x < \frac{t}{a}. \tag{29}$$

Proof. The sign bracket and continuity give a root. Strict monotonicity on $(1, \infty)$ gives uniqueness, including a bracket whose left endpoint is one. The root has $v > 1$, so $w = -v < -1$ selects the minus-one rather than principal real branch. At the root, $x = t/v = e^{-v}$ belongs to $(0, 1/e)$ and satisfies $-x \log x = t$. The reciprocal map is decreasing on the positive interval, giving the stated bounds. □

The implementation tests the sign of $h_t(v)$ with the exact logarithm enclosure. If the resulting interval straddles zero, it increases the logarithm order rather than guessing the sign from a midpoint. An unresolved sign at the configured budget raises an explicit error. The initial bracket starts at $v = 1$ and doubles its upper endpoint until a positive sign is proved; exact bisection then shrinks the bracket to the requested absolute width.

11.4 A concrete checked result

For $t = 1/100$ and a requested 40-bit absolute width, the produced certificate encloses w by

$$-\frac{7116891513251}{1099511627776} < w < -\frac{56935132106001}{8796093022208}. \tag{30}$$

The corresponding small inverse satisfies

$$\frac{274877906944}{177922287831275} < x < \frac{2199023255552}{1423378302650025}. \tag{31}$$

The rational verifier recomputes the endpoint signs using 17 logarithm-series terms and checks that the v, hence w, interval width is at most 2^{-40}. The decimal value displayed in Section 4 is consistent with this enclosure but is not substituted for it.

The executed tests also construct and verify certificates for $t = 1/3$, $1/10$, and $1/100000$, and reject invalid inputs such as nonpositive rationals, $t = 1$, and an inexact Python float. The numerical oracle is evaluated at higher precision for an independent containment check. The exact verifier does not call that oracle.

```
from fractions import Fraction
from lambert_certificate import enclose_lambert_minus_one

c = enclose_lambert_minus_one(Fraction(1, 100), bits=40)
assert c.verify()
print(c.w_interval)
print(c.small_entropy_inverse_interval)
```

The requested bit count controls absolute interval width, not relative accuracy of every derived observable. For the inverse interval, the exact width transformation is

$$\frac{t}{a} - \frac{t}{b} = \frac{t(b-a)}{ab}. \tag{32}$$

This makes it straightforward to choose a stricter v width when a particular absolute x width is required.

11.5 Trust and integration boundary

The proof depends on exact rational arithmetic, the logarithm tail formula, and the monotonicity argument. The Python implementation is not formally verified, and its certificate object is a typed in-process artifact, not a hardened external serialization format. A separate checker for arbitrarily supplied certificate records would need explicit schema and resource validation.

For package integration, one possible first use is strictly as a regression oracle: obtain the real nonprincipal value from the candidate Mathics path and compare it to the rational interval, while also checking branch inequalities. A later narrow provider could return an interval or an explicitly rounded value with a known enclosure. It should not silently install approximate real values into the exact symbolic series algebra.

This work is a concrete refinement of the older quantitative-certificate direction, not a claim that the earlier reviews failed to suggest certificates. Its new role is to verify the precise branch and argument-order boundary exposed by the Mathics port.

12 Development recommendations that follow from the new evidence

12.1 First priority: repair proof and evidence boundaries

The first change set should be small. Make source invalidity sticky, check the copied suite, reject invalid deadlines before process creation, and cover executed workflow dependencies in the path trigger. These changes are locally testable and do not require a broad rewrite of the series algebra. The archive supplies the runner patch and focused tests; the workflow change is a short configuration edit.

The Lambert conversion work should be handled as a separate change set, because it changes the host special-function boundary. Characterize the existing interpreter first, then integrate the complete conversion/arity candidate or make unsupported nonprincipal operations fail closed. Verify exact and inexact requests, reverse conversion, and public exact-core use separately. Do not infer success from the principal-branch workaround.

No conclusion here requires weakening the package's realness checks or turning an unresolved domain into a guessed real branch. Those checks are useful safeguards, especially while portability work is still incomplete.

12.2 An adapter manifest should describe semantics, not merely names

A small machine-readable manifest for the fifteen Mathics modules would make the compatibility surface reviewable. An entry should identify the exact overload, its domain, evaluation and holding behavior, argument permutation if any, explicit refusal policy, runtime version tuple, and the regression IDs that establish the intended behavior.

For example, a `ProductLog` entry would not simply say "available." It would distinguish one-argument principal, two-argument exact, two-argument inexact, reverse SymPy conversion, and branch-dependent numerical behavior. A `FirstPosition` entry would distinguish correct pattern/default behavior from bounded traversal. A `Limit` entry would state its default direction explicitly.

The manifest should record whether a helper is selected by early context binding or late downvalue transformation. That distinction is important for maintenance: a new call site can bypass a finite list of late rewrites even though the adapter exists. The manifest can support a source check for unadapted call sites without forcing an immediate global abstraction layer.

This is more specific than the already proposed public scale-capability registry. It describes the evaluator compatibility ABI: argument order, arity, hold attributes, control-flow destination, and semantic admission. These are exactly the boundaries that a function-name compatibility table misses.

12.3 Version handling should follow capabilities and verified failures

The current adapters are motivated by behavior in Mathics 10.0.1. Some are compensations for interpreter defects rather than portable alternative algorithms. When the interpreter changes, an unconditional workaround can become obsolete, redundant, or harmful. A future version policy

should therefore be based on the exact adapter contract and its tests, not merely a substring test that the runtime name contains "Mathics."[7, 9, 2]

A conservative first implementation could declare a tested version range and warn or refuse particular unverified adapters outside it. A more flexible implementation could use small capability probes, provided those probes have no lasting side effects and do not themselves assume the broken semantics they are trying to test. Neither approach should remove current guards merely because a version number increased.

There should also be a clear ownership decision for each workaround. A generic ProductLog conversion error and an all-matches Position implementation are good upstream candidates. A package-specific real-germ proof helper can remain local. Avoid duplicating the whole symbolic evaluator inside the package when a narrow upstream repair would eliminate the dependency mismatch.

12.4 Resource work should start with the newly exposed dimensions

The next performance change should address amplitude degree and support before adding another broad cache. For the sparse polynomial family, record expanded support size, maximum degree, number of dense positions materialized, and coefficient-expression size. For first-position traversal, record visited nodes and retained matches. These counters identify the source of work without conflating it with interpreter startup or proof timeouts.

A candidate sparse amplitude backend should be accepted only after exact coefficient and remainder equivalence on small instances. Then compare cold and warm runs at matched requested and achieved precision, preserving refusals and reporting peak memory. A claim such as "two rules instead of $N + 1$ slots" is already justified by representation structure; a claim such as "ten times faster" requires actual Mathics measurements that this review did not perform.

The earlier performance register remains the place for generic refinement, convolution, and provenance-size work. The additional goal here is to prevent new compatibility adapters from undoing sparsity before those mature algorithms run.

12.5 Two bounded feature opportunities

The defining-Taylor adapter restricts hypergeometric series to $p \leq q + 1$ and positive real lower parameters, with a constant outer factor and a vanishing monomial argument. That is a defensible sufficient convergence domain, not the full set of exact terminating cases. A negative-integer upper parameter can terminate a hypergeometric series even when the generic nonterminating convergence test would fail. A bounded extension could detect a proved termination degree, check denominator singularities through that degree, and return an exact polynomial.[13]

This is an implementation opportunity at the new provider boundary, not a demonstrated public failure: the existing fallback may already handle a particular terminating input. The benefit should be measured by independent exact coefficient tests and by cases that the fallback actually leaves unresolved. Parameter dependence on the expansion variable must remain excluded unless a new theorem and implementation explicitly support it.

The second opportunity is a sparse magnitude envelope for Fourier amplitudes. Once nonzero amplitude coefficients are represented sparsely, the envelope can sum their absolute values times the appropriate logarithmic powers without constructing all missing degrees. This can improve both resource predictability and the separation between degree and support. It must retain symbolic-real assumptions and preserve all nonzero coefficients; "small numerically" is not an exact zero test.

These opportunities are deliberately narrower than the existing long-range proposals for arbitrary transseries, multivariate implicit systems, complex sectors, and uniform asymptotics. Those directions remain important, but repeating them would not meet the incremental-review objective.

12.6 Acceptance order

A practical order of work is: first, the tested Python runner repairs and workflow dependency coverage; second, exact Lambert conversion and its branch-specific acceptance; third, explicit internal limit-direction policy; fourth, sparse coefficient/amplitude operations and bounded match traversal. Broader parameter support should follow only after the port has reliable acceptance evidence for its present domains.

Each change set should carry its own evidence scope. The runner tests need no Wolfram kernel. The ProductLog bridge needs interpreter integration. The public core regression needs package execution. A sparse-coefficient optimization needs mathematical equivalence tests and separate performance measurements. Combining these into a single green badge without their individual meanings would recreate the evidence ambiguity that R2 exposes.

13 Artifacts and reproduction

13.1 What is included

Artifact	Purpose and status
`article/asymptotic-review.tex`, `article/asymptotic-review.pdf`	This article and its compiled PDF.
`fixtures/run_mathics_tests_upstream.py`	Exact upstream portable-runner fixture for the pinned revision.
`code/stage_runner_patch.py`	Strict-context, separate-output candidate repair; exercised by focused Python tests.
`code/test_runner_audit.py`	Ten executed runner-test methods using explicitly synthetic child processes.
`code/productlog_bridge.py`	Candidate exact/reverse/numerical argument bridge; independently tested, not integrated into Mathics.
`code/stage_productlog_patch.py`, `code/test_productlog_staging.py`	Separate-source staging tool and three executed synthetic-source tests.
`code/lambert_certificate.py`	Exact-rational minus-one branch reference and verifier.
`code/test_independent.py`	Nine executed independent semantic, rational, and structural checks.
`code/MathicsSparseHelpers.wl`	Unexecuted sparse univariate coefficient helper, isolated in an audit context.
`code/ProbeReview.wl`	Unexecuted fresh-process kernel characterization cases; prints observations rather than inventing pass results.

Artifact	Purpose and status
`evidence/`	Actual test logs, observations, novelty ledger, source coverage, and execution scope.

No checksum files, interpreter binaries, fonts, or full copy of the repository are included. The upstream runner naturally contains its own fingerprinting implementation; that is source code needed for the reproduction, not a separate checksum deliverable.

13.2 Run the Python checks

From the archive root, use a Python environment with the versions in `code/requirements.txt`:

```
python -m pip install -r code/requirements.txt
python code/test_independent.py
python code/test_productlog_staging.py
python code/test_runner_audit.py
```

The last script runs bounded synthetic processes and tests the upstream and staged candidate runners. It does not install or invoke Mathics. The supplied logs are the actual results from Python 3.13.5/Linux. The synthetic launcher uses a POSIX executable shebang, so Linux or WSL is the reproduction environment for that test driver. Different operating systems can produce different low-level timeout exception text; the candidate's argument rejection should remain platform-independent.

The runner patch can be staged without changing the fixture:

```
python code/stage_runner_patch.py \
  fixtures/run_mathics_tests_upstream.py \
  candidate/run_mathics_tests.py
```

For use in a repository checkout, stage the candidate into an equivalent validation layout or review the diff before applying it to the canonical source. The runner locates its suite and root relative to its own path, so placing it in an arbitrary directory is not a complete runnable checkout.

The ProductLog staging script likewise requires the actual Mathics source file and a distinct output path. Its syntax checks are not permission to overwrite an installed interpreter without an integration run. The archive intentionally does not redistribute a patched Mathics tree.

13.3 Run a remaining kernel characterization

Obtain a checkout of the pinned repository and use a fresh process for each case. In PowerShell, for example:

```
$env:ASYMPTOTIC_REVIEW_SOURCE =
  (Resolve-Path .\Asymptotic\src\Kernel\AsymptoticAnalysis.wl).Path
$env:ASYMPTOTIC_REVIEW_CASE = "core-real-branch"
python -m mathics --quiet --no-readline --file .\code\ProbeReview.wl
```

The driver also has cases named `productlog-principal-exact`, `productlog-minus-one-exact`, `productlog-minus-one-inexact`, `limit-default`, `sparse-helper`, and `fourier-amplitude`.

An official-kernel control can run the same file using its script interface. The source path should be switched to the generated standalone entry point for the second distribution check.

These calls were not executed in this review. They print runtime identity and observations and deliberately avoid a fake MUnit success count. An automated acceptance wrapper should additionally impose an operating-system process timeout and compare the results against the independent mathematical expectations described above.

13.4 Build the article

Run `article/build.sh` from an environment with pdfLaTeX, the New PX text/math packages, and the listed standard LaTeX packages. Three passes resolve the table of contents and references. The source uses no network fonts or external images. The PDF in the archive was compiled and visually inspected separately from the code tests; typography validation does not contribute to the 22-test code count.

14 Conclusion

The repository's most valuable abstraction is not merely a series coefficient generator. It is the separation of a finite expression, a scale, a branch, and the evidence that permits subsequent operations. The custom real-germ calculus and the native wrapper can coexist usefully when those contracts remain explicit.

The new Mathics port extends that abstraction across a second evaluator, but the evaluator boundary is part of the mathematics. An exact Lambert core cannot be treated as reliable simply because its printed expression contains the expected special-function name. Likewise, a portable acceptance record cannot claim unchanged inputs by forgetting a mutation it already observed.

The actionable outcome is therefore specific: retain invalidating evidence monotonically; validate and snapshot what is actually executed; repair both directions and both arities of the Lambert bridge; keep branch checks independent; prevent sparse adapters from densifying by unrelated degree; and state internal limit directions explicitly. These changes advance the current code without reissuing the extensive backlog already collected in the previous reviews.

A Novelty crosswalk

ID	Nearest prior topic	Incremental contribution
R1	Mathics guide's nonprincipal numerical warning; X05 certificate direction	Exact SymPy conversion, reverse conversion, and inherited numerical arity are separately traced. Adds a branch-sensitive public candidate, complete method-level repair, and exact-rational reference.
R2	V01/V02 test infrastructure	New post-wave-three runner forgets *detected* source changes and fails to check the suite it actually executes. Exact upstream Python reproductions and focused repair are supplied.
R3	General bounded-execution policy	New runner's float-domain validation is insufficient for its process API. NaN, infinity, and huge finite witnesses are executed; fixed before launch.

ID	Nearest prior topic	Incremental contribution
R4	V01 CI	Existing Mathics workflow omits direct command dependencies from its own path trigger. Not a proposal to create CI.
R5	P06/P08; C01 native dense export	New coefficient shim densifies sparse support by degree; Fourier amplitude cost separates degree from frequency count; Mathics Position lacks the naive result-count optimization. No claimed measured speedup.
R6	D06 direction conventions; W3 Taylor ingress	Specific private Limit omitted-option mismatch, with independent separating example. Public wrong-result reachability remains open.

The crosswalk establishes a mechanism-level distinction, not proof that no sentence in any earlier article resembles a proposed test or general principle. The complete machine-readable ledger in the archive records the evidence type and the remaining integration obligation for each row.

B Source-reading map

All fifteen Mathics adapter modules were read in full: `MathicsCompatibility.wl`, `MathicsTimeBudget.wl`, `MathicsCalls.wl`, `MathicsAlgebra.wl`, `MathicsAssumptions.wl`, `MathicsTaylor.wl`, `MathicsSimplification.wl`, `MathicsCalculus.wl`, `MathicsCoreFunctions.wl`, `MathicsSpecialFunctions.wl`, `MathicsInverseBranches.wl`, `MathicsCertificate.wl`, `MathicsRefinement.wl`, `MathicsLists.wl`, and `MathicsFormatting.wl`.

The selected shared-source inspections focused on the entry point's initialization and forward/inverse operations, complete exact-core construction, the Lambert model and inverse construction, source-coordinate reconstruction, Fourier polynomial normalization and convolution, and the first 165 lines of native dispatch. Some long tool responses were truncated beyond those inspected functions; the report does not count unseen tails as reviewed. The portable Python runner and workflow were read in full.

The Mathics 10.0.1 source inspection covered the ProductLog class, the relevant SympyFunction/MPMathFunction methods, the reverse conversion dispatch, and Position's evaluator. These tagged sources are the basis for interpreter-level conclusions, not an assumption that an unspecified future Mathics version behaves the same way.

References

[1] VladimirReshetnikov/Asymptotic, *README*, reviewed revision. Pinned repository overview.

[2] AsymptoticAnalysis, *Kernel entry point and public usage contracts*. Pinned source.

[3] AsymptoticAnalysis, *Kernel module map*. Pinned source map.

[4] AsymptoticAnalysis, *Code review status and complete finding crosswalk*. Pinned register.

[5] AsymptoticAnalysis, *Wave 3 intake*. Pinned intake and crosswalk.

[6] GitHub repository comparison, `6687962f3c85` to `7d1bc832895c`. Pinned comparison.

[7] AsymptoticAnalysis, *Mathics compatibility, scope and validation*. Pinned guide.

[8] AsymptoticAnalysis, *Mathics/Wolfram preservation record*. Pinned historical evidence; source/run scopes are retained in the original record.

[9] AsymptoticAnalysis, *MathicsCompatibility.wl*. Pinned source; module exits, association helpers, FirstPosition and symbol bootstrap.

[10] AsymptoticAnalysis, *MathicsCoreFunctions.wl*. Pinned source; ProductLog construction-site adaptations.

[11] AsymptoticAnalysis, *MathicsAlgebra.wl*. Pinned source.

[12] AsymptoticAnalysis, *MathicsSimplification.wl*. Pinned source; simplification, series and limit adapters.

[13] AsymptoticAnalysis, *MathicsTaylor.wl*. Pinned source; bounded defining-series providers.

[14] AsymptoticAnalysis, *CorePerturbation.wl*. Pinned exact-core construction and result contracts.

[15] AsymptoticAnalysis, *SourceCoordinates.wl*. Pinned source; retained exact carriers and reconstruction.

[16] AsymptoticAnalysis, *FourierCoefficients.wl*. Pinned source; polynomial amplitudes, budgets and inverse construction.

[17] AsymptoticAnalysis, *NativeCompatibility.wl*. Pinned native dispatch and representation boundary.

[18] AsymptoticAnalysis, *run_mathics_tests.py*. Pinned portable runner; exact fixture reproduced in the archive.

[19] AsymptoticAnalysis, *Mathics compatibility workflow*. Pinned workflow configuration.

[20] Mathics3, version 10.0.1, *Exponential integral and special functions*. ProductLog class source.

[21] Mathics3, version 10.0.1, *Builtin infrastructure*. SympyFunction and MPMathFunction source.

[22] Mathics3, version 10.0.1, *SymPy conversion*. Forward and reverse conversion dispatch.

[23] Mathics3, version 10.0.1, *List element operations*. Position evaluator source.

[24] Wolfram Research, *Series*. `https://reference.wolfram.com/language/ref/Series.html`. Reference consulted for expansion forms and order conventions.

[25] Wolfram Research, *SeriesData*. `https://reference.wolfram.com/language/ref/SeriesData.html`.

[26] Wolfram Research, *Asymptotic*. `https://reference.wolfram.com/language/ref/Asymptotic.html`.

[27] Wolfram Research, *InverseSeries*. `https://reference.wolfram.com/language/ref/InverseSeries.html`.

[28] Wolfram Research, *AsymptoticSolve*. https://reference.wolfram.com/language/ref/AsymptoticSolve.html.en.

[29] Wolfram Research, *DiscreteAsymptotic*. https://reference.wolfram.com/language/ref/DiscreteAsymptotic.html.

[30] Wolfram Research, *AsymptoticIntegrate*, *AsymptoticSum*, *AsymptoticDSolveValue*. https://reference.wolfram.com/language/ref/AsymptoticIntegrate.html; https://reference.wolfram.com/language/ref/AsymptoticSum.html; https://reference.wolfram.com/language/ref/AsymptoticDSolveValue.html.

[31] Wolfram Research, *FirstPosition*. https://reference.wolfram.com/language/ref/FirstPosition.html.

[32] Wolfram Research, *Limit*. https://reference.wolfram.com/language/ref/Limit.html; default and explicit direction semantics.

[33] SymPy, *Elementary functions*, LambertW reference. https://docs.sympy.org/latest/modules/functions/elementary.html. Independent experiments used SymPy 1.14.0.

[34] mpmath, *Powers and logarithms*, lambertw reference. https://mpmath.org/doc/current/functions/powers.html. Independent experiments used mpmath 1.3.0.